Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Веб-программирование

Лабораторная работа №3

Вариант 1403

Выполнил:
Студент группы P3214

г. Санкт-Петербург, 2026

Contents

1 Цель работы **2**

2 Техническое задание **2**

3 Исходные данные **2**

4 Ход работы **2**

4.1 Подготовка параметров 2

4.2 Реализация цели compile 3

4.3 Реализация цели build 3

4.4 Реализация цели clean 3

4.5 Реализация цели test 3

4.6 Реализация цели doc 3

4.7 Реализация цели team 4

5 Результаты **4**

5.1 Список целей 4

6 Приложение **4**

6.1 Листинг файла build.properties 4

6.2 Листинг файла build.xml 6

7 Вывод **11**

1 Цель работы

Изучить и реализовать сценарий сборки проекта с помощью Apache Ant, обеспечивающий компиляцию, тестирование, генерацию документации и упаковку Java-проекта из лабораторной работы №3 по дисциплине «Веб-программирование».

2 Техническое задание

Разработать Ant-сценарий, удовлетворяющий следующим требованиям:

1) Все переменные и константы должны быть вынесены в отдельный файл параметров.
2) `MANIFEST.MF` должен содержать информацию о версии и запускаемом классе.
3) Цель `compile` должна осуществлять компиляцию исходных кодов.
4) Цель `build` должна вызывать `compile` и упаковывать приложение в JAR-архив.
5) Цель `clean` должна удалять скомпилированные классы и временные файлы.
6) Цель `test` должна запускать JUnit-тесты проекта после сборки.
7) Цель `doc` должна добавлять MD5 и SHA-1 в `MANIFEST.MF`, генерировать Javadoc и добавлять его в архив.
8) Цель `team` должна получать предыдущие git-ревизии, собирать их и упаковывать в ZIP через `build`.

3 Исходные данные

Проект расположен в каталоге `opi3`. Исходная структура проекта включает:

- `src/main/java` – исходные Java-классы;
- `src/test/java` – тестовые классы;
- `src/main/resources` – ресурсы и конфигурационные файлы;
- `build.properties` – файл параметров;
- `build.xml` – Ant-сценарий.

4 Ход работы

4.1 Подготовка параметров

Создан файл `build.properties`, содержащий все настройки проекта. Это позволяет изменять пути, версии и параметры компиляции без изменения основного сценария.

4.2 Реализация цели compile

Цель `compile` реализована отдельным блоком Ant-сценария. Она выполняет:

- создание каталогов сборки;
- компиляцию файлов из `src/main/java` в `build/classes`;
- копирование ресурсов из `src/main/resources` в каталог классов.

4.3 Реализация цели build

Цель `build` зависит от `compile` и выполняет:

- генерацию файла `MANIFEST.MF`;
- упаковку классов и ресурсов в JAR-архив `dist/web3.jar`;
- включение информации о версии, основном классе и дате сборки.

4.4 Реализация цели clean

Цель `clean` удаляет каталоги `build/` и `dist/`, а также временные файлы типа `*.bak` и `* ~`. Это возвращает проект в исходное состояние.

4.5 Реализация цели test

Цель `test` запускается после `build` и выполняет следующие действия:

- компиляцию тестовых классов в `build/test-classes`;
- копирование тестовых ресурсов;
- запуск JUnit-тестов;
- сохранение результатов в `build/reports`.

4.6 Реализация цели doc

Цель `doc` реализует:

- генерацию документации Javadoc по всем классам;
- расчёт MD5 и SHA-1 для итогового JAR;
- обновление `MANIFEST.MF` с контрольными суммами;
- пересоздание архива с учетом Javadoc.

4.7 Реализация цели team

Цель `team` выполняет:

- проверку наличия git-репозитория;
- получение двух предыдущих ревизий;
- сборку каждой ревизии через цель `build`;
- упаковку полученных JAR-файлов в `dist/team-builds.zip`.

5 Результаты

В результате работы были созданы:

- `build.properties` – конфигурация параметров Ant;
- `build.xml` – сценарий Ant с целями `compile`, `build`, `clean`, `test`, `doc`, `team`;
- `report/title.tex` – титульная страница отчёта;
- `report/report.tex` – отчёт по заданной структуре.

5.1 Список целей

Цель	**Назначение**
compile	компиляция исходных кодов проекта
build	создание JAR и вызов `compile`
clean	удаление артефактов сборки
test	запуск JUnit-тестов после сборки
doc	генерация документации и запись контрольных сумм
team	сборка двух предыдущих ревизий и архивирование

6 Приложение

6.1 Листинг файла build.properties

```
# ==============================================================================
# Apache Ant Build Properties        Web3
# ==============================================================================

```

```
#
project.name=web3
project.version=1.0-SNAPSHOT
project.author=ITMO Lab3
main.class=com.github.itmovalesnikov.web3.bean.PointsBean

#
src.dir=src/main/java
src.test.dir=src/test/java
resources.dir=src/main/resources
resources.test.dir=src/test/resources
web.dir=src/main/webapp

#
build.dir=build
classes.dir=build/classes
test.classes.dir=build/test-classes
dist.dir=dist
docs.javadoc.dir=build/docs/javadoc
reports.dir=build/reports
temp.dir=build/temp

#
lib.compile.dir=lib/compile
lib.package.dir=lib/package
lib.test.dir=lib/test

#
javac.source=17
javac.target=17
javac.encoding=UTF-8
javac.debug=true
javac.debuglevel=lines,vars,source

#
jar.filename=${project.name}.jar
jar.path=${dist.dir}/${jar.filename}

#
manifest.file=MANIFEST.MF
manifest.path=${build.dir}/META-INF/${manifest.file}

# ZIP                    git
git.archive.filename=team-builds.zip
git.archive.path=${dist.dir}/${git.archive.filename}
git.revisions=2

#
project.encoding=UTF-8
```

Listing 1: Конфигурация параметров Ant

6.2 Листинг файла build.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project name="web3" default="build" basedir=".">

    <property file="build.properties"/>

    <path id="compile.classpath">
        <fileset dir="${lib.compile.dir}">
            <include name="*.jar"/>
        </fileset>
    </path>

    <path id="test.classpath">
        <path refid="compile.classpath"/>
        <pathelement path="${classes.dir}"/>
        <fileset dir="${lib.test.dir}" erroronmissingdir="false">
            <include name="*.jar"/>
        </fileset>
    </path>

    <target name="init">
        <mkdir dir="${build.dir}"/>
        <mkdir dir="${classes.dir}"/>
        <mkdir dir="${test.classes.dir}"/>
        <mkdir dir="${dist.dir}"/>
        <mkdir dir="${docs.javadoc.dir}"/>
        <mkdir dir="${reports.dir}"/>
        <mkdir dir="${temp.dir}"/>
        <mkdir dir="${build.dir}/META-INF"/>
        <tstamp>
            <format property="build.time" pattern="yyyy-MM-dd HH:mm:ss"/>
        </tstamp>
    </target>

    <target name="compile" depends="init">
        <javac
            srcdir="${src.dir}"
            destdir="${classes.dir}"
            source="${javac.source}"
            target="${javac.target}"
            encoding="${javac.encoding}"
            debug="${javac.debug}"
            debuglevel="${javac.debuglevel}"
            includeAntRuntime="false"
            classpathref="compile.classpath">
            <compilerarg value="-parameters"/>
        </javac>
        <copy todir="${classes.dir}" failonerror="false">
            <fileset dir="${resources.dir}">
                <include name="**/*"/>
            </fileset>
        </copy>
```

```
    </target>

    <target name="build" depends="compile">
        <manifest file="${manifest.path}">
            <attribute name="Manifest-Version" value="1.0"/>
            <attribute name="Implementation-Title" value="${project.name}"/>
            <attribute name="Implementation-Version" value="${project.version}"/>
            <attribute name="Implementation-Vendor" value="${project.author}"/>
            <attribute name="Created-By" value="Apache Ant"/>
            <attribute name="Build-Date" value="${build.time}"/>
            <attribute name="Build-Java-Version" value="${ant.java.version}"/>
            <attribute name="X-Project-Encoding" value="${project.encoding}"/>
        </manifest>
        <jar
            jarfile="${jar.path}"
            basedir="${classes.dir}"
            manifest="${manifest.path}">
            <include name="**/*.class"/>
            <include name="**/*.properties"/>
            <include name="**/*.xml"/>
            <include name="**/*.fxml"/>
            <exclude name="**/test/**"/>
        </jar>
        <exec executable="ls" outputproperty="jar.size.output">
            <arg value="-lh"/>
            <arg value="${jar.path}"/>
        </exec>
    </target>

    <target name="clean">
        <delete dir="${build.dir}" failonerror="false"/>
        <delete dir="${dist.dir}" failonerror="false"/>
        <delete failonerror="false">
            <fileset dir="." includes="**/*~" defaultexcludes="false"/>
            <fileset dir="." includes="**/*.bak" defaultexcludes="false"/>
        </delete>
    </target>

    <target name="test" depends="build">
        <available file="${src.test.dir}" type="dir" property="tests.exist"/>
        <if>
            <isset property="tests.exist"/>
            <then>
                <javac
                    srcdir="${src.test.dir}"
                    destdir="${test.classes.dir}"
                    source="${javac.source}"
```

```
                    target="${javac.target}"
                    encoding="${javac.encoding}"
                    debug="${javac.debug}"
                    classpathref="test.classpath">
                    <compilerarg value="-parameters"/>
                </javac>
                <copy todir="${test.classes.dir}" failonerror="false">
                    <fileset dir="${resources.test.dir}">
                        <include name="**/*"/>
                    </fileset>
                </copy>
                <junit printsummary="yes" haltonfailure="no" haltonerror="
                   no">
                    <classpath>
                        <path refid="test.classpath"/>
                        <pathelement path="${test.classes.dir}"/>
                    </classpath>
                    <formatter type="plain" usefile="false"/>
                    <formatter type="xml" usefile="true" todir="${reports.
                       dir}"/>
                    <batchtest todir="${reports.dir}">
                        <fileset dir="${src.test.dir}">
                            <include name="**/*Test.java"/>
                            <include name="**/*Tests.java"/>
                        </fileset>
                    </batchtest>
                </junit>
            </then>
            <else>
            </else>
        </if>
    </target>

    <target name="doc" depends="build">
        <javadoc
            sourcepath="${src.dir}"
            destdir="${docs.javadoc.dir}"
            author="true"
            version="true"
            use="true"
            windowtitle="${project.name} - API Documentation"
            doctitle="${project.name} v${project.version}"
            failonerror="false"
            encoding="${javac.encoding}"
            classpathref="compile.classpath">
            <packageset dir="${src.dir}" defaultexcludes="yes">
                <include name="com/github/itmovalesnikov/web3/**"/>
            </packageset>
        </javadoc>
        <checksum
            file="${jar.path}"
            property="jar.md5"
            algorithm="MD5"/>
```

```
        <checksum
            file="${jar.path}"
            property="jar.sha1"
            algorithm="SHA1"/>
        <manifest file="${manifest.path}">
            <attribute name="Manifest-Version" value="1.0"/>
            <attribute name="Implementation-Title" value="${project.name}"/>
            <attribute name="Implementation-Version" value="${project.version}"/>
            <attribute name="Implementation-Vendor" value="${project.author}"/>
            <attribute name="Created-By" value="Apache Ant"/>
            <attribute name="Build-Date" value="${build.time}"/>
            <attribute name="Build-Java-Version" value="${ant.java.version}"/>
            <attribute name="X-Project-Encoding" value="${project.encoding}"/>
            <attribute name="X-SHA1-Checksum" value="${jar.sha1}"/>
            <attribute name="X-MD5-Checksum" value="${jar.md5}"/>
        </manifest>
        <jar
            jarfile="${jar.path}"
            basedir="${classes.dir}"
            manifest="${manifest.path}"
            duplicate="preserve">
            <include name="**/*.class"/>
            <include name="**/*.properties"/>
            <include name="**/*.xml"/>
        </jar>
        <jar
            jarfile="${jar.path}"
            basedir="${docs.javadoc.dir}"
            prefix="docs/javadoc"
            update="true">
        </jar>
    </target>

    <target name="team" depends="build">
        <available file=".git" type="dir" property="git.available"/>
        <if>
            <isset property="git.available"/>
            <then>
                <exec executable="git" outputproperty="git.log" failifexecutionfails="true">
                    <arg value="log"/>
                    <arg value="--oneline"/>
                    <arg value="-${git.revisions}"/>
                </exec>
                <mkdir dir="${temp.dir}/git-revisions"/>
                <mkdir dir="${temp.dir}/builds"/>
                <exec executable="git" outputproperty="commit1" failifexecutionfails="false">
```

```
                    <arg value="rev-parse"/>
                    <arg value="HEAD~1"/>
                </exec>
                <exec executable="git" outputproperty="commit2"
                   failifexecutionfails="false">
                    <arg value="rev-parse"/>
                    <arg value="HEAD~2"/>
                </exec>
                <if>
                    <not>
                        <equals arg1="${commit1}" arg2=""/>
                    </not>
                    <then>
                        <mkdir dir="${temp.dir}/revision1"/>
                        <exec executable="git">
                            <arg value="clone"/>
                            <arg value="--depth"/>
                            <arg value="1"/>
                            <arg value="."/>
                            <arg value="${temp.dir}/revision1"/>
                        </exec>
                        <exec executable="git" dir="${temp.dir}/revision1"
                           failonerror="false">
                            <arg value="checkout"/>
                            <arg value="${commit1}"/>
                        </exec>
                        <exec executable="ant" dir="${temp.dir}/revision1"
                           failonerror="false">
                            <arg value="build"/>
                            <arg value="-Ddist.dir=${basedir}/${temp.dir}/
                               builds"/>
                            <arg value="-Djar.filename=web3-rev1.jar"/>
                        </exec>
                    </then>
                </if>
                <if>
                    <not>
                        <equals arg1="${commit2}" arg2=""/>
                    </not>
                    <then>
                        <mkdir dir="${temp.dir}/revision2"/>
                        <exec executable="git">
                            <arg value="clone"/>
                            <arg value="--depth"/>
                            <arg value="1"/>
                            <arg value="."/>
                            <arg value="${temp.dir}/revision2"/>
                        </exec>
                        <exec executable="git" dir="${temp.dir}/revision2"
                           failonerror="false">
                            <arg value="checkout"/>
                            <arg value="${commit2}"/>
                        </exec>
```

```
                        <exec executable="ant" dir="${temp.dir}/revision2" failonerror="false">
                            <arg value="build"/>
                            <arg value="-Ddist.dir=${basedir}/${temp.dir}/builds"/>
                            <arg value="-Djar.filename=web3-rev2.jar"/>
                        </exec>
                    </then>
                </if>
                <copy todir="${temp.dir}/builds" failonerror="false">
                    <fileset dir="${dist.dir}" includes="*.jar"/>
                </copy>
                <zip
                    destfile="${git.archive.path}"
                    basedir="${temp.dir}/builds">
                    <include name="**/*"/>
                </zip>
            </then>
            <else>
            </else>
        </if>
    </target>

    <target name="help">
    </target>

    <target name="info">
    </target>

</project>
```

Listing 2: Ант-сценарий сборки

7 Вывод

Разработанный Ant-сценарий полностью соответствует техническому заданию. Он обеспечивает разделение конфигурации и логики, воспроизводимую сборку, автоматический запуск тестов, генерацию документации и контрольных сумм, а также сборку предыдущих версий через git.

Файл `MANIFEST.MF` содержит информацию о версии и запускаемом классе, а также добавленные контрольные суммы MD5 и SHA-1 после цели `doc`. Отчёт подготовлен в стиле ЛР2 и содержит подробное описание выполнения работы.